Universidad Nacional Mar del Plata - Facultad de Ingeniería
Departamento de Informática – Tecnologías Informáticas B

Guía de Trabajos Prácticos N° 2

Modalidad Tutoría

Para realizar esta guía será necesaria la instalación de un servidor de base de datos relacional (RDBMS). Un SGBDR (en español, Sistema de Gestión de Base de Datos Relacionales) es un software que permite crear, actualizar y administrar bases de datos relacionales (ver Guía de Trabajos Prácticos N°1). La primera parte de esta guía repite la creación de usuario y base de datos de la Guía 1.

Parte 1 Práctica Asistida

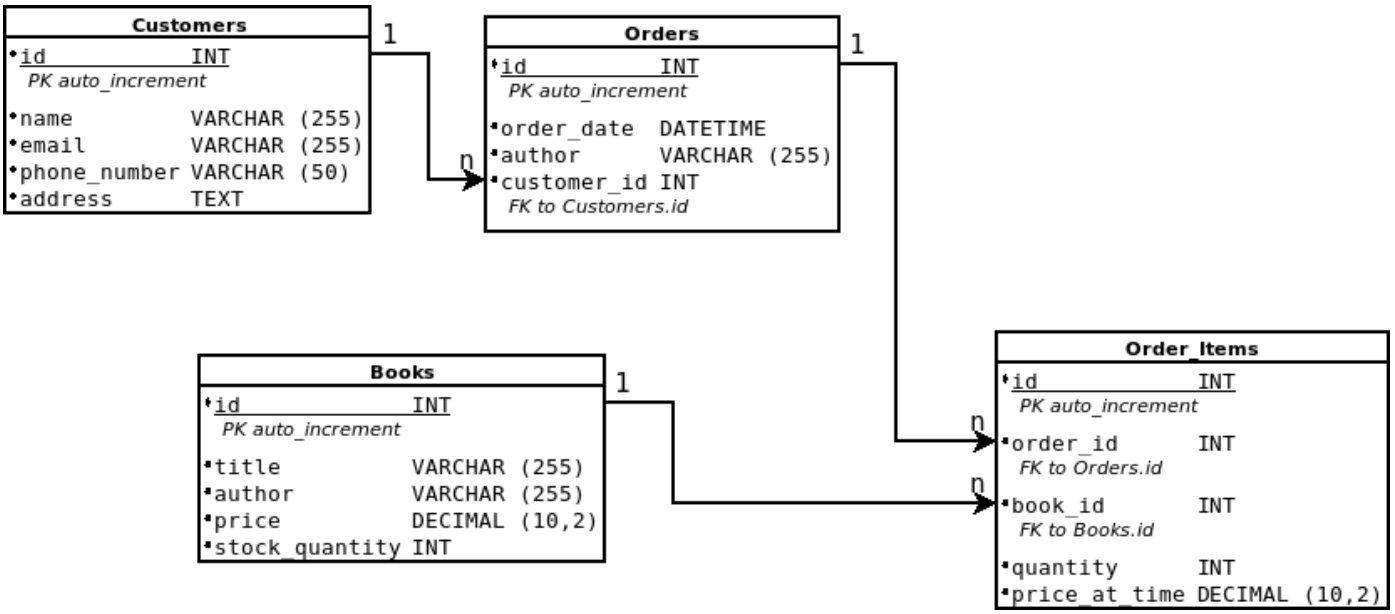
1. Ejemplo didáctico: Base de datos para una tienda de libros
    a. Analice el siguiente modelo de la Base de Datos

(Listado y detalle de tablas)

- **Customers (Clientes)**
    - **id** (int, PK, auto_increment)
    - name (varchar)
    - email (varchar)
    - phone_number (varchar)
    - address (text)
- **Books (Libros)**
    - **id** (int, PK, auto_increment)
    - title (varchar)
    - author (varchar)
    - price (decimal)
    - stock_quantity (int)
- **Orders (Pedidos)**
    - **id** (int, PK, auto_increment)
    - order_date (datetime)
    - **customer_id** (int, FK hacia Customers.id)
- **Order_Items (Detalles/Ítems de los pedidos)**
    - **id** (int, PK, auto_increment)
    - **order_id** (int, FK hacia Orders.id)

- **book_id** (int, FK hacia Books.id)
- quantity (int)
- price_at_time (decimal)

b. Compare con el Diagrama de Base de Datos:

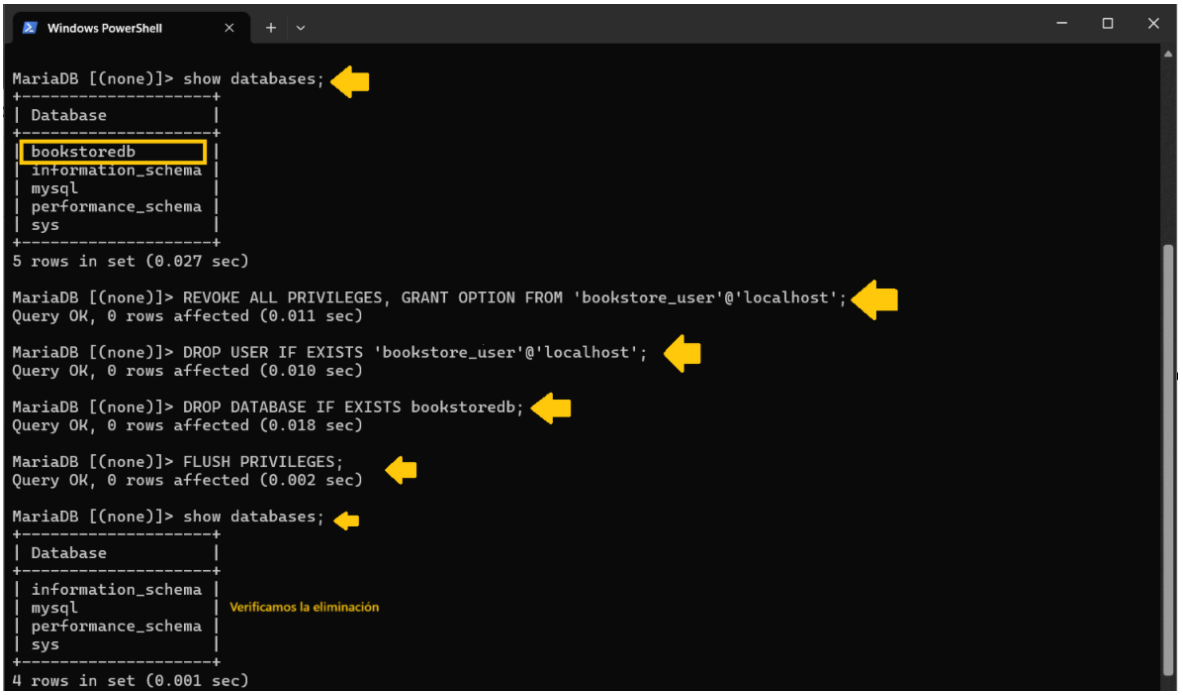


Identifique los elementos presentes en el diagrama y en la descripción de la base de datos bookstoredb. Observe las convenciones de uso de mayúscula y minúscula en los nombres de tablas, tipos y campos. Analice los valores de almacenamiento asociados a cada tipo. Tome apunte de lo identificado.

2. Lenguaje de Definición de Datos (DDL)Creación de la base de datos, credenciales y creación de tablas (Script SQL). Ejecute las siguientes sentencias y revise los resultados en Adminer:

a. Volver a estado cero si existiesen usuarios y base de datos creadas previamente con el mismo nombre

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'bookstore_user'@'localhost';
DROP USER IF EXISTS 'bookstore_user'@'localhost';
DROP DATABASE IF EXISTS bookstoredb;
FLUSH PRIVILEGES;
```



b. Crear la Base de Datos (EJECUTAR COMO ROOT)

Crea una base de datos llamada **`bookstoredb`** con el cotejamiento **`utf8_unicode_ci`**.

```
CREATE DATABASE bookstoredb CHARACTER SET utf8 COLLATE utf8_unicode_ci;
```

---

b. Crear un Usuario en MySQL (EJECUTAR COMO ROOT)

Ahora creamos un usuario llamado **`bookstore_user`** con la contraseña **`12345 (a los fines prácticos usamos una contraseña insegura y fácil de recordar)`**:

```
CREATE USER 'bookstore_user'@'localhost' IDENTIFIED BY '12345';
```

---

c. Asignar Permisos al Usuario (EJECUTAR COMO ROOT)

Otorgamos todos los permisos sobre la base de datos **`bookstoredb`** al usuario **`bookstore_user`**:

```
GRANT ALL PRIVILEGES ON bookstoredb.* TO 'bookstore_user'@'localhost';
```

Para que los cambios surtan efecto, ejecutamos:

```sql
FLUSH PRIVILEGES;
```

---

d. Verificar los Permisos Asignados (EJECUTAR COMO ROOT)

Podemos verificar qué permisos tiene **`bookstore_user`** con:

```sql
SHOW GRANTS FOR 'bookstore_user'@'localhost';
```

Esto mostrará algo como:

```sql
GRANT ALL PRIVILEGES ON `bookstoredb`.* TO 'bookstore_user'@'localhost'
```

---

d. Salir del Usuario Root

Ahora salimos de la sesión actual en MySQL (si hubiéramos ejecutado esto en una terminal, sino no es necesario):

```sql
EXIT;
```

---

e. Iniciar Sesión con el Nuevo Usuario

Ahora iniciamos sesión en MySQL con **`bookstore_user`** `(solo en una terminal, en adminer este proceso es en entorno gráfico)`:

```sql
mysql -u bookstore_user -p
```

Cuando pida la contraseña, ingresamos:

```sql
12345
```

`A partir de aquí crear las tablas:`

```sql
-- 5 Usar la base de datos bookstore (esto solo es necesario
-- cuando usamos la terminal)
USE bookstoredb;

-- Crear la tabla Customers
CREATE TABLE Customers (
```

```sql
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    email VARCHAR(255) UNIQUE,
    phone_number VARCHAR(50),
    address TEXT
);

-- Crear la tabla Books
CREATE TABLE Books (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255),
    author VARCHAR(255),
    price DECIMAL(10, 2),
    stock_quantity INT
);
-- Crear la tabla Orders
CREATE TABLE Orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_date DATETIME,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES Customers(id)
);
-- Crear la tabla Order_Items
CREATE TABLE Order_Items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT,
    book_id INT,
    quantity INT,
    price_at_time DECIMAL(10, 2),
    FOREIGN KEY (order_id) REFERENCES Orders(id),
    FOREIGN KEY (book_id) REFERENCES Books(id)
);
```

Ejecutar estas sentencias como usuario bookstore_user: Insertar y actualizar Datos (Lenguaje de Manipulación de datos DML).

```sql
-- Insertar clientes
INSERT INTO Customers (name, email, phone_number, address) VALUES
('John Doe', 'john.doe@example.com', '555-1234', '123 Main St'),
('Jane Smith', 'jane.smith@example.com', '555-5678', '456 Oak Ave');

-- Insertar libros
INSERT INTO Books (title, author, price, stock_quantity) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 10.99, 100),
('1984', 'George Orwell', 8.99, 50);
```

```sql
-- Insertar pedidos
INSERT INTO Orders (order_date, customer_id) VALUES
(NOW(), 1),
(NOW(), 2);

-- Insertar detalles de los pedidos
INSERT INTO Order_Items (order_id, book_id, quantity, price_at_time)
VALUES
(1, 1, 2, 10.99),
(2, 2, 1, 8.99);
```

3. Lenguaje de Consulta de Datos (DQL): Ejemplos de consultas básicas para recuperar información de las tablas, ejecute las siguientes sentencias y revise los resultados en Adminer, haga la misma prueba en la terminal:
   a. Obtener todos los clientes:
      - `SELECT * FROM Customers;`

```
MariaDB [bookstoredb]> SELECT * FROM Customers;
+----+------------+-----------------------+--------------+-------------+
| id | name       | email                 | phone_number | address     |
+----+------------+-----------------------+--------------+-------------+
|  1 | John Doe   | john.doe@example.com  | 555-1234     | 123 Main St |
|  2 | Jane Smith | jane.smith@example.com | 555-5678     | 456 Oak Ave |
+----+------------+-----------------------+--------------+-------------+
```

   b. Obtener todos los pedidos realizados por un cliente específico:
      - `SELECT * FROM Orders WHERE customer_id = 1;`

   c. Obtener todos los libros que están en un pedido específico:
      -
```sql
SELECT b.title, oi.quantity, oi.price_at_time
FROM Order_Items AS oi
JOIN Books AS b ON oi.book_id = b.id
WHERE oi.order_id = 1;
```

   d. Obtener el total de un pedido (usando la cantidad de libros y el precio en el momento de la compra):
      -
```sql
SELECT SUM(oi.quantity * oi.price_at_time) AS total
FROM Order_Items AS oi
```

```
WHERE oi.order_id = 1;
```

4. Otros ejemplos DML. Nuevamente ejecute las siguientes sentencias y revise los resultados en Adminer, haga la misma prueba en la terminal, (probablemente deba ajustar alguna condición para probar las mismas sentencias, lea el apartado DATO IMPORTANTE):

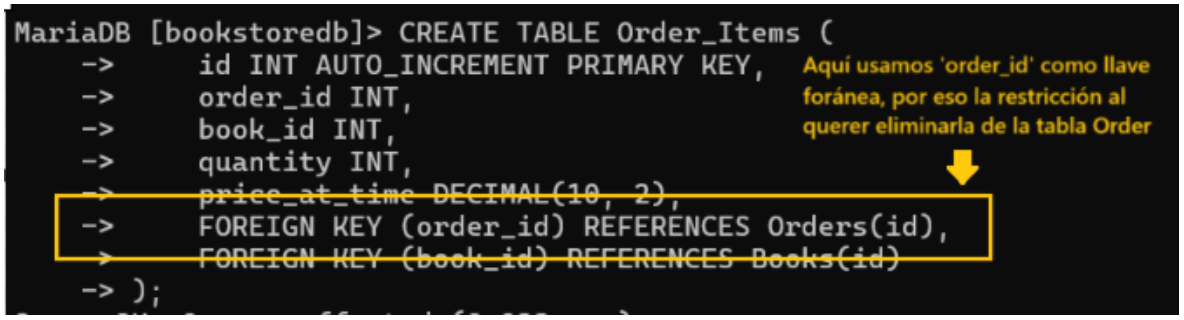
   a. Actualizar el stock de un libro:

- `UPDATE Books SET stock_quantity = stock_quantity - 1 WHERE id = 1;`

   b. Eliminar un pedido:

- `DELETE FROM Orders WHERE id = 2;`

DATO IMPORTANTE: Al eliminar van a obtener un error-> Cannot delete or update a parent row: a foreign key constraint fails (`bookstoreDB`.`Order_Items`, CONSTRAINT `Order_Items_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `Orders` (`id`))



Eso es correcto, ya que es la función de la restricción de la clave foránea. Hay formas de que se permita el borrado y la actualización cuando existe esta restricción. Esto se realiza utilizando las restricciones combinadas con las opciones para actualizar y eliminar en cascada:

ON DELETE CASCADE ON UPDATE CASCADE

Entonces para poder eliminar en cascada modificamos la tablas correspondientes del siguiente modo:

```
ALTER TABLE `Orders`

DROP FOREIGN KEY `Orders_ibfk_1`,
```

```sql
ADD FOREIGN KEY (`customer_id`) REFERENCES `Customers` (`id`) ON DELETE CASCADE
ON UPDATE CASCADE;
```

```sql
ALTER TABLE `Order_Items`

DROP FOREIGN KEY `Order_Items_ibfk_1`,

ADD FOREIGN KEY (`order_id`) REFERENCES `Orders` (`id`) ON DELETE CASCADE ON
UPDATE CASCADE;
```

Nota: ibfk = *InnoDB Foreign Key*. Es una convención de nomenclatura.